

P1072R9

basic_string::resize_and_overwrite

Published Proposal, 2021-08-14

This version:

<http://wg21.link/P1072R9>

Authors:

[Chris Kennelly](#) (Google)

[Mark Zeren](#) (VMware)

Audience:

LEWG, LWG, SG16

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Optimized writing of data into a basic_string.

Table of Contents

1 Motivation

2 Proposal

3 Implementation

4 Examples

4.1 Stamping a Pattern

4.2 Interacting with C

5 Design Considerations

5.1 Method vs. Alternatives

5.2 Allocator Interaction

5.3 Undefined Behavior

5.4 Bikeshedding

6 Alternatives Considered

6.1 Tag Type

6.2 Non-Lambda Approach

6.3 Standalone Type: `storage_buffer`

6.4 Externally-Allocated Buffer Injection

7 Related Work

7.1 Google

7.2 MongoDB

7.3 VMware

7.4 Discussion on std-proposals

7.5 DynamicBuffer

7.6 P1020R1

7.7 Boost

7.8 Thrust

8 Wording

8.1 `[version.syn]`

8.2 `[basic.string.general]`

8.3 `[string.require]`

8.4 `[string.capacity]`

8.5 `[container.requirements.general]`

9 History

9.1 R8 → R9

9.2 R7 → R8

9.3 R6 → R7

9.4 R5 → R6

9.5 R4 → R5

9.6 R3 → R4

9.7 R2 → R3

9.8 R1 → R2

9.9 R0 → R1

10 Acknowledgments

References

Informative References

§ 1. Motivation

Performance sensitive code is impacted by the cost of initializing and manipulating strings. When writing data into a `basic_string`, a programmer is faced with an unhappy choice:

- **Pay for extra initialization** — `resize`, which zero initializes, followed by copy.
- **Pay for extra copies** — Populate a temporary buffer, copy it to the string.
- **Pay for extra "bookkeeping"** — `reserve` followed by small appends, each of which checks capacity and null terminates.

The effects of these unhappy choices can all be measured at scale, yet C++'s hallmark is to leave no room between the language (or in this case the library) and another language.

LEWGI polled on this at the [SAN] Meeting:

We should promise more committee time to [\[P1072R1\]](#), knowing that our time is scarce and this will leave less time for other work?

Unanimous consent

LEWG at the [SAN] Meeting:

We want to solve this problem

SF	F	N	A	SA
17	5	0	0	0

Willing to solve this for string without solving it for vector

SF	F	N	A	SA
6	9	2	1	0

§ 2. Proposal

This proposal addresses the problem by adding `basic_string::resize_and_overwrite`:

```
template<class Operation>
constexpr void resize_and_overwrite(size_type n, Operation op);
```

7. *Effects*: Alters the value of `*this` as follows:

- If `n <= size()`, erases the last `size() - n` elements.
- If `n > size()`, appends `n - size()` default-initialized elements.
- Invokes `erase(begin() + op(data(), n), end())`.

8. *Remarks*:

If `op` throws the behavior is undefined.

Let `o = size()` before the call to `resize_and_overwrite`.

Let `m = op(data(), n)`.

`m <= n` otherwise the behavior is undefined.

If `m > o`, `op` shall replace ([**expr.ass**]) the values stored in the character array [`data() + o`, `data() + m`). Until replaced, the values may be indeterminate [**basic.indet**] [*Note: `*(data() + o)` may not be `charT()`. - end note*]

`op` may write to `data() + n + 1`. Any value written will be replaced with `charT()` after `op` returns. [*Note: This facilitates interoperability with functions that write a trailing null. - end note*]

When `op` is called `*this` is in an unspecified state. `op` shall not bind or in any way access `*this`.

`op` shall not allow its first argument to be accessible after it returns.

In order to enable `resize_and_overwrite`, this proposal makes it implementation-defined whether `basic_string` uses `allocator_traits::construct` and `allocator_traits::destroy` to construct and destroy the "char-like objects" that it controls. See § 5.2 [Allocator Interaction](#) and § 8 [Wording](#) below for more details.

§ 3. Implementation

`libc++` includes a private implementation of a prior version of this proposal (based on an earlier revision) and uses it to avoid a dynamic allocation in `std::filesystem` [LIBC++].

§ 4. Examples

§ 4.1. Stamping a Pattern

Consider writing a pattern several times into a string:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
    std::string ret;

    ret.reserve(pattern.size() * count);
    for (size_t i = 0; i < count; i++) {
        // SUB-OPTIMAL:
        // * Writes 'count' nulls
        // * Updates size and checks for potential resize 'count' times
        ret.append(pattern);
    }

    return ret;
}
```

Alternatively, we could adjust the output string's size to its final size, avoiding the bookkeeping in `append` at the cost of extra initialization:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
    std::string ret;

    const auto step = pattern.size();
    // SUB-OPTIMAL: We memset step*count bytes only to overwrite them.
    ret.resize(step * count);
    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(ret.data() + i * step, pattern.data(), step);
    }
}
```

```

    return ret;
}

```

With this proposal:

```

std::string GeneratePattern(const std::string& pattern, size_t count) {
    std::string ret;

    const auto step = pattern.size();
    // GOOD: No initialization
    ret.resize_and_overwrite(step * count, [&](char* buf, size_t n) {
        for (size_t i = 0; i < count; i++) {
            // GOOD: No bookkeeping
            memcpy(buf + i * step, pattern.data(), step);
        }
        return step * count;
    });

    return ret;
}

```

§ 4.2. Interacting with C

Consider wrapping a C API while working in terms of C++'s `basic_string` vocabulary. We *anticipate* over-allocating, as computation of the *length* of the data is done simultaneously with the computation of the *contents*.

```

extern "C" {
    int compress(void* out, size_t* out_size, const void* in, size_t in_size)
    size_t compressBound(size_t bound);
}

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // SUB-OPTIMAL: Extra initialization
    compressed.resize(size);
}

```

```

    int ret = compress(&*compressed.begin(), &size, input.data(), input.size());
    if (ret != OK) {
        throw ...some error...
    }

    // Set actual size.
    compressed.erase(size);
    return compressed;
}

```

With this proposal:

```

extern "C" {
    int compress(void* out, size_t* out_size, const void* in, size_t in_size);
    size_t compressBound(size_t bound);
}

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t bound = compressBound(input.size());
    int ret;

    // GOOD: No initialization
    compressed.resize_and_overwrite(bound, [&](char* buf, size_t n) {
        size_t compressed_size = n;
        ret = compress(buf, &out_size, input.data(), input.size());
        return compressed_size;
    });
    if (ret != OK) {
        throw ...some error...
    }
    return compressed;
}

```

§ 5. Design Considerations

§ 5.1. Method vs. Alternatives

During the [SAN] Meeting LEWG expressed a preference for implementing this functionality as a new method on `basic_string` (as proposed in [P1072R0]) rather than a standalone "storage buffer" type (one option in [P1072R1]):

Method on string vs storage_buffer type:

Strong Method	Method	Neutral	Type	Strong Type
9	2	3	2	2

During the [KOA] Meeting, LEWG expressed a preference for not weakening `basic_string`'s invariants after `resize_and_overwrite` returns. Under this revision:

- A default-initialized buffer is *only* accessible to `op`.
- Under penalty of undefined behavior, `op` must assign (replace) the values in resulting buffer. The default initialized portion (`[data()+m, data()+n)`) above is erased and inaccessible to the program without further violation of class invariants.
- This proposal follows the "lambda" option discussed in [SAN].

Several other alternatives were discussed at [SAN], [KOA], and on the reflector. Please see § 6 [Alternatives Considered](#) below for more details.

§ 5.2. Allocator Interaction

Unlocking the desired optimizations requires *some* change to `basic_string`'s interaction with allocators. This proposal does what we think is the simplest possible change: remove the requirement that `basic_string` call `allocator_traits::construct` or `allocator_traits::destroy`.

This restriction should be acceptable because `basic_string` is defined to only hold "non-array trivial standard-layout" types. [strings.general] p1:

This Clause describes components for manipulating sequences of any non-array trivial standard-layout (6.7) type. Such types are called *char-like types*, and objects of *char-like types* are called *char-like objects* or simply *characters*.

Removing calls to `construct` and `destroy` is compatible with `pmr::basic_string` as long as `uses_allocator_v<charT>` is `false`, which should be the case in practice.

Along the way, this proposal clarifies ambiguous language in `[string.require]` and `[container.requirements.general]` by:

- Stating explicitly that `basic_string` is allocator-aware.
- Introducing the term "allocator-aware" earlier in `[container.requirements.general]`.
- Not attempting to mention other "components" in `[container.requirements.general]`. The other allocator-aware containers (just `basic_string` and `regex::match_results`?) can independently state that they are "allocator-aware" in their own clauses.

libstdc++ and msvc allow strings of non-trivial type. That might force those libraries to continue to support `construct` and `destroy` as "extensions". On the other hand, libc++ disallows non-trivial `charTs`, so any such extensions are non-portable. See godbolt.org.

§ 5.3. Undefined Behavior

`resize_and_overwrite` exposes users to UB if they read indeterminate (`[basic.indet]`) values in the string buffer passed to `op`. Despite this foot-gun, `resize_and_overwrite` is appealing because:

- Uninitialized buffers are not uncommon in C++. See for example `new T[20]` or the recently adopted `make_unique_for_overwrite`.
- Dynamic analyzers like Memory Sanitizer [MSan] and Valgrind [Valgrind] can catch uninitialized reads.

The version of this proposal seen by LEWG in [Cologne] throws `std::range_error` when `op` returns a value greater than `n`. The authors now recommend and LEWG confirmed via [LEWGNovTelecon] to use UB. For the `op` to have written greater than `n` bytes, it would have had to overrun the provided buffer (already UB).

The authors further recommend UB if `op` were to throw. `resize_and_overwrite` is motivated by measured performance improvements that are small individually but significant at scale. Providing exception guarantees would incur overhead that is in tension with the original motivation.

§ 5.4. Bikeshedding

What do we want to call this method?

- `resize_and_overwrite` (as proposed). This is consistent with `make_unique_for_overwrite` [P1973R1]. Unlike the `make_unique` version, this concludes with the buffer being overwritten (as far as `op` specifies), rather than requiring further action be taken.
- `resize_default_init` (as proposed). This is consistent with `make_unique_default_init` [P1020R1], adopted in San Diego. This name was removed from the working paper subsequently and renamed `make_unique_for_overwrite`.
- `resize_uninitialized` (as proposed in R0). "Uninitialized" is different from "default-initialized", which is what we want to specify. Also, `uninitialized` is already used elsewhere in the Library to mean "not constructed at all", as in `uninitialized_copy`; so this would be an overloading of the term.

§ 6. Alternatives Considered

§ 6.1. Tag Type

At the [SAN] Meeting, LEWG showed support for a tag argument type:

Approval (vote for as many as you find acceptable)

13	Go back to <code>resize_uninitialized</code>
15	Do tag type (<code>default_initialize</code>) for <code>c'tor</code> / <code>resize()</code>
12	Continue with <code>storage_buffer</code> (as per R2 of this paper)
7	Crack open with a lambda
7	RAII separate type

For example:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
    const auto step = pattern.size();

    // GOOD: No initialization
    std::string ret(step * count, std::string::default_init);
    for (size_t i = 0; i < count; i++) {
        // GOOD: No bookkeeping
        memcpy(ret.data() + i * step, pattern.data(), step);
    }
}
```

```
    return ret;
}
```

Benefits:

- A constructor that takes a tag type simplifies some use cases, like the example above.
- Matches existing practice in Boost. See [§ 7.7 Boost](#).

Drawbacks:

- Feature creep / complexity — A tag type invites generalizing for all allocator-aware container types. We agree that this is desirable and even has implementation experience in Boost. However default initialization is not enough. There is also "implicit construction" (see [\[P1010R1\]](#), [\[P0593R2\]](#)) and "relocation" (see [\[P1144R0\]](#), [\[P1029R1\]](#)). Neither of these features are yet in the language. It is too early to generalize. Note that the second poll quoted in [§ 1 Motivation](#) shows support for solving this problem for `[basic_]string` but not `vector`.
- In reflector discussion of `make_unique_default_init` [Zero], there was a preference for avoiding tag types. The standard library has `copy_backward`, not `copy` with a tag, and `count_if`, rather than `count` with a predicate.

Conclusion:

LEWG should explore tags for allocator-aware containers, but that work should not block near-term enablement of efficient `[std::]string` builders.

§ 6.2. Non-Lambda Approach

In [\[P1072R0\]](#) and [\[P1072R3\]](#) of this proposal, the authors considered a method `resize_default_init` / `resize_uninitialized` which left a default-initialized buffer accessible to users of the `basic_string` instance after the method returned. This method was rejected in [KOA], due to the weakened class invariants.

For illustration:

```
extern "C" {
    int compress(void* out, size_t* out_size, const void* in, size_t in_size)
    size_t compressBound(size_t bound);
}
```

```

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // GOOD: No initialization
    compressed.resize_default_init(size);
    int ret = compress(&*compressed.begin(), &size, input.data(), input.size());
    if (ret != OK) {
        throw ...some error...
    }

    // Suppose size is the value of size before the call to compress and size'
    // is the value of size after the call to compress.
    //
    // If size' < size, then:
    //   std::cout << compressed[size' + 1]
    // ...triggers a read of uninitialized data.

    // Set actual size.
    compressed.erase(size);
    return compressed;
}

```

§ 6.3. Standalone Type: `storage_buffer`

In [P1072R1], we considered `storage_buffer`, a standalone type providing a `prepare` method (similar to the `resize_uninitialized` method proposed here) and `commit` (to promise, under penalty of UB, that the buffer had been initialized).

At the [SAN] Meeting, this approach received support from LEWGI in light of the [post-Rapperswil] email review indicating support for a distinct type. This approach was rejected by the larger LEWG room in San Diego Meeting Diego.

The proposed type would be move-only.

```

std::string GeneratePattern(const std::string& pattern, size_t count) {
    std::storage_buffer<char> tmp;

    const auto step = pattern.size();

```

```

// GOOD: No initialization
tmp.prepare(step * count + 1);
for (size_t i = 0; i < count; i++) {
    // GOOD: No bookkeeping
    memcpy(tmp.data() + i * step, pattern.data(), step);
}

tmp.commit(step * count);
return std::string(std::move(tmp));
}

```

For purposes of the container API, `size()` corresponds to the *committed* portion of the buffer. This leads to more consistency when working with (and explicitly copying to) other containers via iterators, for example:

```

std::storage_buffer<char> buf;
buf.prepare(100);
*fill in data*
buf.commit(50);

std::string a(buf.begin(), buf.end());
std::string b(std::move(buf));

assert(a == b);

```

Benefits:

- By being move-only, we would not have the risk of copying types with trap representations (thereby triggering UB).
- Uninitialized data is only accessible from the `storage_buffer` type. For an API working with `basic_string`, no invariants are weakened. Crossing an API boundary with a `storage_buffer` is much more obvious than a "`basic_string` with possibly uninitialized data." Uninitialized bytes (the promise made by `commit`) never escape into the valid range of `basic_string`.

Drawbacks:

- `storage_buffer` requires introducing an extra type to the standard library, even though its novel functionality (from `string` and `vector`) is limited to the initialization abilities.

- `basic_string` is often implemented with a short-string optimization (SSO) and an extra type would need to implement that (likely by additional checks when moving to/from the `storage_buffer`) that are often unneeded.

§ 6.4. Externally-Allocated Buffer Injection

In [P1072R1], we considered that `basic_string` could "adopt" an externally `allocator::allocate`'d buffer. At the [SAN] Meeting, we concluded that this was:

- **Not critical**
- **Not constrained in the future**
- **Overly constraining to implementers.** Allowing users to provide their own buffers runs into the "offset problem". Consider an implementation that stores its `size` and `capacity` inline with its data, so `sizeof(container) == sizeof(void*)`.

```
class container {  
    struct Rep {  
        size_t size;  
        size_t capacity;  
    };  
  
    Rep* rep_;  
};
```

If using a `Rep`-style implementation, the mismatch in offsets requires an $O(N)$ move to shift the contents into place and trigger a possible reallocation.

- **Introducing new pitfalls.** It would be easy to mix `new[]` and `allocator::allocate` inadvertently.

§ 7. Related Work

§ 7.1. Google

Google has a local extension to `basic_string` called `resize_uninitialized` which is wrapped as `STLStringResizeUninitialized`.

- [Abseil] uses this to avoid bookkeeping overheads in `StrAppend` and `StrCat`.

- [Snappy]
 - In [decompression](#), the final size of the output buffer is known before the contents are ready.
 - During [compression](#), an upperbound on the final compressed size is known, allowing data to be efficiently added to the output buffer (eliding [append](#)'s checks) and the string to be shrunk to its final, correct size.
- [Protobuf] avoids extraneous copies or initialization when the size is known before data is available (especially during parsing or serialization).

§ 7.2. MongoDB

MongoDB has a string builder that could have been implemented in terms of [basic_string](#) as a return value. However, as explained by Mathias Stearn, zero initialization was measured and was too costly. Instead a custom string builder type is used:

E.g.: <https://github.com/mongodb/mongo/blob/master/src/mongo/db/fts/unicode/string.h>

```
/**
 * Strips diacritics and case-folds the utf8 input string, as needed to sup
 * options.
 *
 * The options field specifies what operations to *skip*, so kCaseSensitive
 * means to skip case folding and kDiacriticSensitive means to skip diacrit
 * striping. If both flags are specified, the input utf8 StringData is retu
 * directly without any processing or copying.
 *
 * If processing is performed, the returned StringData will be placed in
 * buffer. buffer's contents (if any) will be replaced. Since we may retur
 * the input unmodified the returned StringData's lifetime is the shorter o
 * the input utf8 and the next modification to buffer. The input utf8 must
 * point into buffer.
 */
static StringData caseFoldAndStripDiacritics(StackBufBuilder* buffer,
                                             StringData utf8,
                                             SubstrMatchOptions options,
                                             CaseFoldMode mode);
```

(Comments re-wrapped.)

§ 7.3. VMware

VMware has an internal string builder implementation that avoids `std::string` due, in part, to `reserve`'s zero-writing behavior. This is similar in spirit to the MongoDB example above.

§ 7.4. Discussion on std-proposals

This topic was discussed in 2013 on std-proposals in a thread titled "Add `basic_string::resize_uninitialized` (or a similar mechanism)":

<https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/XIO4KbBTxl0>

§ 7.5. DynamicBuffer

The [\[N4734\]](#) (the Networking TS) has *dynamic buffer* types.

§ 7.6. P1020R1

See also [\[P1020R1\]](#) "Smart pointer creation functions for default initialization". Adopted in San Diego.

§ 7.7. Boost

Boost provides a related optimization for vector-like containers, introduced in [\[SVN r85964\]](#) by Ion Gaztañaga.

E.g.: [boost/container/vector.hpp](#):

```
//! <b>Effects</b>: Constructs a vector that will use a copy of allocator a
//!    and inserts n default initialized values.
//!
//! <b>Throws</b>: If allocator_type's allocation
//!    throws or T's default initialization throws.
//!
//! <b>Complexity</b>: Linear to n.
//!
//! <b>Note</b>: Non-standard extension
vector(size_type n, default_init_t);
```



```
vector(size_type n, default_init_t, const allocator_type &a)
...
void resize(size_type new_size, default_init_t);
...
```

These optimizations are also supported in Boost Container's `small_vector`, `static_vector`, `deque`, `stable_vector`, and `string`.

§ 7.8. Thrust

The Thrust library has "a RAII-type `thrust::detail::temporary_array` which has a vector-like interface and a constructor with a tag parameter that indicates its elements should not be initialized." - [Bryce Adelstein Leibach].

E.g. `thrust/thrust/detail/temporary_array.inl`:

```
template<typename T, typename TemporaryArray, typename Size>
__host__ __device__
typename thrust::detail::disable_if<
    avoid_initialization<T>::value
>::type
    construct_values(TemporaryArray &a,
                    Size n)
{
    a.default_construct_n(a.begin(), n);
} // end construct_values()
```

§ 8. Wording

Wording is relative to [\[N4892\]](#).

Motivation for some of these edits can be found in [§ 5.2 Allocator Interaction](#).

§ 8.1. [version.syn]

In [version.syn], add:

```
#define __cpp_lib_string_resize_and_overwrite YYYYMMML // also in <string>
```

Adjust the placeholder value as needed so as to denote this proposal's date of adoption.

§ 8.2. [basic.string.general]

In [basic.string.general], in the synopsis, add `resize_and_overwrite`:

```
...
// 21.3.3.5, capacity
constexpr size_type size() const noexcept;
constexpr size_type length() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr void resize(size_type n, charT c);
constexpr void resize(size_type n);
template<class Operation>
constexpr void resize_and_overwrite(size_type n, Operation op);
constexpr size_type capacity() const noexcept;
constexpr void reserve(size_type res_arg);
constexpr void shrink_to_fit();
constexpr void clear() noexcept;
[[nodiscard]] constexpr bool empty() const noexcept;
```

§ 8.3. [string.require]

Add a note to [string.require]:

3. In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT`. Every object of type `basic_string<charT, traits, Allocator>` uses an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used is obtained as described in 22.2.1. In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall meet the character traits requirements (21.2).

[Note 1: Every specialization `basic_string<charT, traits, Allocator>` is an allocator-aware container, but does not use the allocator's `construct` and `destroy` member functions. ([**container.requirements.general**]). — *end note*]

[*Note 1 2:* The program is ill-formed if `traits::char_type` is not the same type as `charT`. — *end note*]

4. References, pointers, and iterators referring to the elements of a `basic_string` sequence may be invalidated by the following uses of that `basic_string` object:

§ 8.4. [**string.capacity**]

Note that [**string.require**] p4 has blanket wording for mutating operations, and [LWG2414] introduces blanket wording for reentrancy.

In `[string.capacity]`:

```
constexpr void resize(size_type n, charT c);
```

5. *Effects*: Alters the value of `*this` as follows:

- If `n <= size()`, erases the last `size() - n` elements.
- If `n > size()`, appends `n - size()` copies of `c`.

```
constexpr void resize(size_type n);
```

6. *Effects*: Equivalent to `resize(n, charT())`.

```
template<class Operation>
```

```
constexpr void resize_and_overwrite(size_type n, Operation op);
```

7. *Let*

— `o = size()` before the call to `resize_and_overwrite`.

— `k` be `min(o, n)`.

— `p` be a `charT*`, such that the range `[p, p + n]` is valid and `this->compare(0, k, p, k) == 0` is true before the call. The values in the range `[p + k, p + n]` may be indeterminate [**basic.indet**].

— `r = op(p, n)`.

8. *Mandates*: `op(p, n)` has an integer-like type ([**iterator.concept.winc**]).

9. *Preconditions*:

— `op(p, n)` does not throw an exception.

— `r ≥ 0`.

— `r ≤ n`.

— After evaluating `op(p, n)` there are no indeterminate values in the range `[p, p + r)`.

10. *Effects*: Evaluates `op(p, n)`, replaces the contents of `*this` with `[p, p + r)`, and invalidates all pointers and references to the range `[p, p + n]`.

11. After `resize_and_overwrite()`, `capacity()` is greater than or equal to `n` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than `n`.

12. *Recommended practice*: Implementations should avoid unnecessary copies and allocations by, for example, making `p` a pointer into internal storage and by restoring `*(p + r)` to `charT()`.

after evaluating `op(p, n)`.

```
constexpr size_type capacity() const noexcept;  
...
```

§ 8.5. [container.requirements.general]

In [container.requirements.general] p3 provide an explicit exception for `basic_string`. While we are here, switch to "allocator-aware" and "elements" terminology.

3. ~~For the components affected by this subclause that declare an `allocator_type`, objects stored in these components shall be constructed~~ Allocator-aware containers (Table 76) other than `basic_string` construct elements using the function `allocator_traits<allocator_type>::rebind_traits<U>::construct` and ~~destroyed~~ destroy elements using the function `allocator_traits<allocator_type>::rebind_traits<U>::destroy` [allocator.traits.members] (20.10.8.3), where `U` is either `allocator_type::value_type` or an internal type used by the container. These functions are called only for the container's element type, not for internal types used by the container.

We provide a similar exception for `basic_string` in p16:

16. Given an allocator type `A` and given a container type `X` having a `value_type` identical to `T` and an `allocator_type` identical to `allocator_traits<A>::rebind_alloc<T>` and given an lvalue `m` of type `A`, a pointer `p` of type `T*`, an expression `v` of type (possibly `const`) `T`, and an rvalue `rv` of type `T`, the following terms are defined. If `X` is not allocator-aware or is a specialization of `basic_string`, the terms below are defined as if `A` were `allocator<T>` — no allocator object needs to be created and user specializations of `allocator<T>` are not instantiated:

§ 9. History

§ 9.1. R8 → R9

- Rebased on [N4892].

- In [**string.capacity**]
 - Added a pointer to [\[LWG2414\]](#).
 - Reworded in terms of Let, Mandates, Preconditions, Effects, and Recommended practice.
 - Added requirements for allocation and capacity.
- In [**container.requirements.general**]
 - Use simple (wording) exceptions for `basic_string`.
 - Use "allocator-aware".
 - Use "element" instead of "object".
- Add only Note in [**string.require**], removing normative changes.
- Wording typos: `typename` -> `class`.
- Fix markdown glitch in R8 history.

§ 9.2. R7 → R8

LEWG reviewed R7 via [April2021Telecon]

POLL: Modify P1072R7 (`basic_string::resize_and_overwrite`) by adding a feature test macro, and send revised paper to electronic polling to be forwarded to LWG with ship vehicle C++23, and priority B2.

SF	F	N	A	SA
9	11	1	0	0

Attendance: 31

Number of Authors: 2

Author Position: Strongly Favor

Outcome: Strong consensus in favor of sending to electronic polling.

LEWG did electronic polling via [D2384R0]

Poll 7: Modify P1072R7 (`basic_string::resize_and_overwrite`) by adding a feature test macro, and then send the revised paper to Library Working Group for C++23, classified as an improvement of an existing feature ([P0592R4] bucket 2 item).

SF	F	N	A	SA
17	8	0	1	1

Outcome: Consensus in favor.

- Added LEWG poll from [April2021Telecon]
- Added feature test macro.
- Fixed ambiguous indentation in `string.capacity` Effects.
- Further fixes for `CompressWrapper` example.
- Rebased on [\[N4885\]](#).
- Added LEWG electronic polling results from [D2384R0].

§ 9.3. R6 → R7

- Rebased on [\[N4878\]](#).
- Fixed `CompressWrapper` example.
- Allowed writing one past the end.
- Reorganized [§ 5.3 Undefined Behavior](#).

§ 9.4. R5 → R6

LEWG reviewed R5 via [LEWGNovTelecon].

POLL: `resize_default_init` should be able to work in `constexpr` context.

No objection to unanimous consent. Attendance: 23

POLL: We prefer UB over throwing an exception if the operation reports it wrote more bytes than it has access to.

No objection to unanimous consent. Attendance: 21

POLL: We want to rename `resize_default_init` for consistency, in a mailing list review.

SF	F	N	A	SA
3	12	0	1	0

Modifications since R5:

- Indicated poll outcomes for UB when `op` reports an invalid value.
- Renamed to `resize_and_overwrite`. This is based on naming discussions for `std::make_unique_default_init` being ultimately renamed to `std::make_unique_for_overwrite`, but differs slightly because the memory accessible has been overwritten. Overwriting the buffer is an integral part of the operation performed.
- Applied `constexpr`

§ 9.5. R4 → R5

A draft of this revision was presented to LEWG at the [Cologne] meeting.

Forward to LWG for C++23

SF	F	N	A	SA
10	5	1	0	0

- Rebased on [\[N4830\]](#).
- Proposed UB instead of `throwing std::range_error`.

§ 9.6. R3 → R4

- Applied feedback from the [KOA] meeting.
- Moved to using a callback routine, rather than leaving invariants weakened from the time `resize_default_init` returned until values in the buffer were assigned.

§ 9.7. R2 → R3

- Applied Jonathan Wakely's editorial comments on [§ 8 Wording](#).
- Rebased on [\[N4791\]](#).

- Editorial changes to [§ 1 Motivation](#) and [§ 6.1 Tag Type](#) to support our design choice.
- Added the reference to [LIBC++] in [§ 3 Implementation](#).

§ 9.8. R1 → R2

Applied feedback from [SAN] Meeting reviews.

- Reverted design to "option A" proposed in [\[P1072R0\]](#).
- Switched from `resize_uninitialized` to `resize_default_init`.
- Added discussion of alternatives considered.
- Specified allocator interaction.
- Added wording.

§ 9.9. R0 → R1

Applied feedback from LEWG [post-Rapperswil] Email Review:

- Shifted to a new vocabulary types: `storage_buffer` / `storage_node`
 - Added presentation of `storage_buffer` as a new container type
 - Added presentation of `storage_node` as a node-like type
- Added discussion of design and implementation considerations.
- Most of [\[P1010R1\]](#) Was merged into this paper.

§ 10. Acknowledgments

Thanks go to **Arthur O'Dwyer** for help with wording and proof reading, to **Jonathan Wakely** for hunting down the language that makes `basic_string` allocator-aware, and to **Glen Fernandes**, **Corentin Jabot**, **Billy O'Neal**, and **Mathias Stearn** for design discussion. Special thanks to **Eric Fiselier** for providing the implementation.

§ References

§ Informative References

[LWG2414]

Stephan T. Lavavej. *Member function reentrancy should be implementation-defined*. Open. URL: <https://wg21.link/lwg2414>

[N4734]

Jonathan Wakely. *Working Draft, C++ Extensions for Networking*. 4 April 2018. URL: <https://wg21.link/n4734>

[N4791]

Richard Smith. *Working Draft, Standard for Programming Language C++*. 26 November 2018. URL: <https://wg21.link/n4791>

[N4830]

Richard Smith. *Committee Draft, Standard for Programming Language C++*. 15 August 2019. URL: <https://wg21.link/n4830>

[N4878]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2020. URL: <https://wg21.link/n4878>

[N4885]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 17 March 2021. URL: <https://wg21.link/n4885>

[N4892]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 18 June 2021. URL: <https://wg21.link/n4892>

[P0593R2]

Richard Smith. *Implicit creation of objects for low-level object manipulation*. 11 February 2018. URL: <https://wg21.link/p0593r2>

[P1010R1]

Mark Zeren, Chris Kennelly. *Container support for implicit lifetime types*. 8 October 2018. URL: <https://wg21.link/p1010r1>

[P1020R1]

Glen Joseph Fernandes, Peter Dimov. *Smart pointer creation with default initialization*. 6 November 2018. URL: <https://wg21.link/p1020r1>

[P1029R1]

Niall Douglas. *[[move_relocates]]*. 7 August 2018. URL: <https://wg21.link/p1029r1>

[P1072R0]

Chris Kennelly, Mark Zeren. *Default Initialization for basic_string*. 4 May 2018. URL: <https://wg21.link/p1072r0>

[P1072R1]

Chris Kennelly, Mark Zeren. *Optimized Initialization for basic_string and vector*. 7 October 2018. URL: <https://wg21.link/p1072r1>

[P1072R3]

Chris Kennelly, Mark Zeren. *basic_string::resize_default_init*. 21 January 2019. URL: <https://wg21.link/p1072r3>

[P1144R0]

Arthur O'Dwyer, Mingxin Wang. *Object relocation in terms of move plus destroy*. 4 October 2018. URL: <https://wg21.link/p1144r0>

[P1973R1]

Nicolai Josuttis. *Rename _default_init functions (NB Comment DE002)*. 12 February 2020. URL: <https://wg21.link/p1973r1>